

SHL Assessment Recommender — Approach Document

Problem Decomposition

The core challenge is converting vague hiring intent into a grounded shortlist without hallucinating

catalog items. I decomposed this into three sub-problems:

1. **Catalog grounding** — how to make every recommendation traceable to a real SHL product
2. **Dialogue management** — when to clarify, when to recommend, when to refuse
3. **Schema reliability** — ensuring the JSON contract is met even when the model misbehaves

Architecture

Stack

- **FastAPI** for the HTTP service (lightweight, async-ready, fast cold starts)
- **Anthropic Claude Sonnet** via direct SDK (no framework overhead; easier to control output)
- **Static catalog JSON** as the retrieval source (no vector DB needed at this catalog size)
- **Render / Railway** for free-tier deployment (Docker-based, health-check friendly)

Catalog Strategy

I scraped the SHL Individual Test Solutions catalog and stored it as `data/catalog.json`.

The full catalog (name, URL, test types, job levels, duration, description) is injected verbatim

into the system prompt. At ~50 assessments, this fits comfortably within a 200k context window

without chunking or vector retrieval.

****Trade-off considered****: RAG with FAISS would scale to thousands of products but adds latency and

a retrieval failure mode. At the current catalog size, in-context retrieval is faster, simpler, and

more reliable. If the catalog grows past ~300 items, moving to semantic search would be warranted.

Prompt Design

The system prompt enforces:

- JSON-only output (no prose outside the schema)
- Explicit instructions for each conversational behavior (clarify, recommend, refine, compare, refuse)
- A compact catalog listing so the model can match assessments to queries without hallucinating

****Key decision****: Asking the model to output raw JSON (not markdown-fenced JSON) and using a regex

fallback parser makes schema compliance robust against minor formatting drift.

Guard-rails (Defense in Depth)

Even if the model produces invalid recommendations, the application layer:

1. Checks every URL against `VALID\_URLS` (the set of scraped catalog URLs)
2. Attempts name-based lookup as a secondary match
3. Silently drops any item that passes neither check
4. Hard-caps the list at 10 items

This means a prompt injection that tries to insert `https://evil.com/fake` will be stripped

before it reaches the response — regardless of what the model outputs.

Stateless Design

Each `POST /chat` carries the full conversation history. The service holds no session state.

Message history is capped at the last 14 messages (7 turns) to stay within the 8-turn evaluator

limit and control token usage.

Conversational Behavior

Behavior	Implementation
----------	----------------

---	---
-----	-----

Clarify	Prompt instructs: do not recommend if query is vague; ask ONE question at a time
----------------	--

Recommend	Prompt instructs: 1–10 items once role, level, and key needs are known
------------------	--

Refine	Full conversation history is sent on every call; model sees prior shortlist and updates it
---------------	--

Compare	Prompt instructs: use only catalog data (description, test types, duration)
----------------	---

Refuse	Prompt enumerates off-topic categories; model returns `recommendations: []`
---------------	---

Evaluation Approach

`eval.py` implements a local harness that:

- Replays 10 conversation traces against the live endpoint
- Reports **schema compliance** (hard pass/fail per response)
- Reports **Mean Recall@10** across traces with labeled expected assessments
- Reports **behavior probe pass-rate** (binary checks: no recommend on vague, refuse off-topic, honor refinements, no fake URLs under injection)

What Didn't Work

- **Asking the model to self-report confidence**: led to long replies that broke JSON output.

Removed; the application layer decides whether to recommend based on `recommendations` being non-empty.

- **Asking multiple clarifying questions at once**: made conversations feel like forms.

Constraining to ONE question per turn improved naturalness and Recall@10 because users gave cleaner answers.

- **Markdown JSON fences in the output**: the model occasionally wrapped JSON in `` `json` `.

Added a regex stripper in the parser to handle this reliably.

AI Tools Used

- **Claude** (Anthropic): prompt iteration, test trace generation, and code review
- All design decisions and code structure are my own and can be defended line-by-line

